

로그인 기능과 구현 방법 설명서

이메일·비밀번호 인증, 아이디 유일성, 계정 찾기·탈퇴까지
wishkit이 Firebase로 계정을 다루는 방식을 한곳에 정리합니다.

대상 코드 wishlist-appversion2 / flutter_app
백엔드 Firebase Auth · Cloud Firestore · Storage
작성일 2026년 8월 14일

이 문서가 다루는 것

회원가입, 이메일 인증, 로그인, 이메일 찾기, 비밀번호 찾기·변경, 로그아웃, 계정 탈퇴, 그리고 아이디가 겹치면 팔로우가 깨지지 않게 한 이유와 구현입니다. 구글·카카오 소셜 로그인은 현재 없습니다.

1. 목차

- 2. 한 줄 요약
- 3. 왜 Firebase를 쓰나
- 4. 역할 나누기 — 화면 / 상태 / 저장소
- 5. 사용자가 보는 화면과 이동 규칙
- 6. 회원가입
- 7. 이메일 인증
- 8. 로그인과 세션 복원
- 9. 이메일 찾기 · 비밀번호 찾기
- 10. 비밀번호 변경 · 로그아웃
- 11. 계정 탈퇴
- 12. 아이디가 겹치면 안 되는 이유
- 13. Firestore 데이터 구조
- 14. 보안 규칙
- 15. 에러 메시지
- 16. 주요 파일
- 17. 아직 없는 것 · 운영 시 확인할 것

2. 한 줄 요약

wishkit 로그인엔 이메일 + 비밀번호만 사용합니다. 누가 로그인했는지는 Firebase Authentication이 증명하고, 이름·아이디·위시리스트·팔로우 같은 앱 데이터는 Cloud Firestore에 둡니다. 프로필 사진과 리뷰 사진은 Firebase Storage에 올립니다.

가입 직후에는 앱에 바로 들어가지 않습니다. Firebase가 보낸 인증 메일의 링크를 누른 뒤에야 Firestore 프로필이 만들어집니다. 아이디(handle, 예: @mina)는 전역에서 하나만 쓸 수 있습니다. 같은 아이디가 두 명에게 붙으면 친구 찾기·팔로우·이메일 찾기가 사람을 구분하지 못합니다.

구분	쓰는 서비스	하는 일
로그인 자격 증명	Firebase Auth	이메일/비밀번호 가입·로그인, 인증 메일, 비밀번호 재설정, 계정 삭제
앱 계정(프로필)	Cloud Firestore	users/ {uid}, handles/ {아이디}, 팔로우, 위시리스트, 알림, 리뷰
파일	Firebase Storage	avatars/ {uid}/... , reviews/ {uid}/...
기기 임시값	SharedPreferences	가입 중 이름·아이디 초안, 알림 on/off, 살까말까 로컬 보완

표 1. 로그인과 계정에 쓰는 저장소

3. 왜 Firebase를 쓰나

우리 앱 서버(파싱 엔진)는 쇼핑 링크에서 상품 정보를 읽는 역할입니다. 회원 시스템·세션·비밀번호를 직접 만들지 않습니다. 대신 Google Firebase 프로젝트 softstudio-wishlist-app에 계정을 맡깁니다.

- Auth가 비밀번호를 해시해 보관합니다. 앱은 비밀번호를 Firestore에 저장하지 않습니다.
- 로그인이 성공하면 Firebase가 uid(사용자 고유 ID)와 ID 토큰을 줍니다. 이후 Firestore 읽기/쓰기는 이 uid로 권한을 검사합니다.

- 앱을 켜도 Firebase SDK가 세션을 유지합니다. 시작 시 currentUser가 있으면 그 세션을 이어갑니다.
- 인증 메일·비밀번호 재설정 메일은 Firebase 콘솔 템플릿을 사용합니다. 문구는 firebase_email_templates.txt 에 적어 두었습니다.

초기 버전과의 차이

wishlist-appversion1 은 “아무 이메일/비밀번호나 넣으면 된다”는 가짜 로그인이었습니다. 지금 version2 는 실제 Firebase 계정입니다. 콘솔에 없는 이메일로는 들어가지 않습니다.

4. 역할 나누기 — 화면 / 상태 / 저장소

코드를 세 층으로 보면 이해가 쉽습니다. 화면은 입력만 받고, 규칙은 AppStore가 정하고, Firebase 호출은 AccountRepository가 합니다.

층	위치	책임
화면	lib/screens/auth/..., mypage_screen.dart	입력 폼, 로딩, 사용자에게 보여줄 한국어 예러
앱 상태	lib/data/app_store.dart	로그인 여부, 인증 대기, 세션 적재, 유효성 검사, 에러를 쉬운 말로 바꿈
저장소	lib/services/account_repository.dart	Auth/Firestore/Storage API, 아이디 예약, 탈퇴 시 데이터 삭제
라우팅	lib/main.dart 의 GoRouter redirect	로그인 전·인증 대기·환영 화면에 따라 강제 이동

표 2. 로그인 코드의 층

핵심 상태 플래그는 세 가지입니다.

- isLoggedIn — 이메일 인증까지 끝나고 Firestore 프로필을 읽은 상태. 이때만 홈으로 갑니다.
- awaitingEmailVerification — Auth 계정은 생겼지만 메일 링크를 아직 안 누른 상태. /verify-email 에 붙잡힙니다.
- showSignupWelcome — 방금 인증을 마친 직후 1.8초 환영 화면(/welcome).

5. 사용자가 보는 화면과 이동 규칙

경로	화면	언제 보이거나
/login	LoginScreen	로그아웃 상태. 같은 화면에서 회원가입으로 전환
/verify-email	EmailVerificationScreen	가입 직후, 또는 미인증 계정으로 로그인했을 때
/account-recovery	AccountRecoveryScreen	로그인 화면의 “이메일 / 비밀번호 찾기”
/welcome	SignupWelcomeScreen	인증 직후 한 번. 자동으로 홈으로 이동
/	위시리스트 홈	isLoggedIn == true

표 3. 인증 관련 경로

redirect 규칙 (GoRouter)

- 인증 대기 중이면 어디를 눌러도 /verify-email 로 보냅니다.
- 로그인되지 않았고 계정 찾기 화면도 아니면 /login 으로 보냅니다. 그래서 홈-마이페이지는 로그인 없이 열리지 않습니다.
- 이미 로그인된 채 /login, /verify-email, /account-recovery, /welcome 에 있으면 홈(/)으로 보냅니다.
- 앱을 다시 켜면 Firebase currentUser를 보고, 미인증이면 다시 인증 화면, 인증됐으면 프로필을 읽어 홈으로 갑니다.

6. 회원가입

로그인 화면에서 “계정이 없나요? 회원가입”을 누르면 이름·아이디 칸이 더 나옵니다. 제출 시 `AppStore.register()` 가 아래 순서로 일합니다.

1. 입력 검사	2. 아이디 예약 확인	3. 초안 저장	4. Auth 가입	5. 인증 화면
이메일·비밀번호·아이디 필수. 비밀번호 6자 이상	handles 와 users.handle Lower 를 조회. 남이 쓰면 거절	이름·아이디를 기기에 잠시 저장 (인증 전에 Firestore에 안 씀)	<code>createUserWithEmailAndPassword</code> + 인증 메일 발송	<code>isLoggedIn</code> 은 false. /verify-email 로 이동

그림 1. 회원가입이 홈으로 바로 들어가지 않는 이유

중요합니다. 이 시점에는 Firestore 프로필이 아직 없습니다. Auth에만 “이 이메일로 가입 중”인 사용자가 생깁니다. 아이디를 지금 확정 예약하지 않는 이유는, 메일을 인증하지 않고 이탈한 사람이 아이디만 선점하는 일을 줄이기 위해서입니다. 대신 가입 직전에 `assertHandleAvailable`으로 한 번 검사하고, 인증이 끝난 뒤 트랜잭션으로 handles 문서를 차지합니다.

아이디 정규화

- 앞뒤 공백 제거, 가운데 공백 제거.
- @ 가 없으면 앞에 @ 를 붙입니다. 사용자는 “mina”만 넣어도 @mina 가 됩니다.
- 비교와 예약 키는 소문자입니다. @Mina 와 @mina 는 같은 아이디로 봅니다.

가입 중 이름·아이디는 어디에 있나

SharedPreferences 키 `pending_register_name` / `pending_register_handle`. 인증 메일을 누르기 전에 앱을 꺼도, 다시 들어와 인증을 마치면 그 이름·아이디로 프로필을 만듭니다. 인증을 취소(돌아가기)하면 이 초안과 Auth 세션을 지웁니다.

7. 이메일 인증

화면은 세 버튼입니다. “인증 완료했어요”, “인증 메일 다시 보내기”, “돌아가기”.

버튼	실제 동작
인증 완료했어요	<code>user.reload()</code> 후 <code>emailVerified</code> 확인. 아직이면 안내. 됐으면 <code>ensureProfile</code> 로 Firestore 계정을 만들고 환영 화면
인증 메일 다시 보내기	Firestore <code>sendEmailVerification()</code> . 스팸함을 보라는 안내가 화면에 있음

버튼	실제 동작
돌아가기	logout + 가입 초안 삭제. 로그인 화면으로. Auth 사용자 자체는 콘솔에 남을 수 있음(미인증 계정)

표 4. 이메일 인증 화면

ensureProfile — 이때 진짜 앱 계정이 생긴다

- emailVerified 가 false 이면 프로필을 만들지 않습니다.
- 이미 users/ {uid} 가 있으면 아이디 예약만 고치고 끝냅니다(재로그인).
- 없으면 reclaimIdentityIfNeeded 로, 같은 이메일의 낡은 Firestore 잔여 문서를 치웁니다. (콘솔에서 Auth 만 지운 경우 대비)
- 트랜잭션으로 handles/ {아이디소문자} 에 uid를 쓰고, users/ {uid} 에 이름·이메일·아이디·아바타를 씁니다.
- 기본 탭 “전체”만 만듭니다. 데모 카테고리는 넣지 않습니다.

인증을 막 마치면 showSignupWelcome = true 가 되어 “회원가입을 축하합니다”가 약 1.8초 보인 뒤 위시리스트로 갑니다.

8. 로그인과 세션 복원

로그인 버튼

- signInWithEmailAndPassword(email, password).
- reload 후 emailVerified 가 false 이면 홈으로 보내지 않고 인증 화면으로 보냅니다. (가입만 하고 메일을 안 누른 사람)
- 인증된 사용자면 _hydrateSession: 프로필 보장 → 탭·상품·친구·알림·리뷰 로드.
- 알림·받은 장바구니·리뷰를 읽을 때 규칙이 아직 안 맞으면 그 부분만 비우고 로그인만 막지 않습니다. 예전에 permission-denied 때문에 로그인 직후 튕기던 문제를 이렇게 막았습니다.

앱 재시작

main() 에서 Firebase.initializeApp 후 AppStore.init(). currentUser 가 있으면 같은 _hydrateSession을 탑니다. firebase_options.dart 의 apiKey 가 YOUR_ 로 시작하면 firebaseReady=false 이고 로그인 버튼이 사실상 동작하지 않으며, FIREBASE_SETUP 안내를 띄웁니다.

9. 이메일 찾기 · 비밀번호 찾기

로그인 화면의 “이메일 / 비밀번호 찾기” → /account-recovery. 한 화면에서 세그먼트로 두 기능을 고릅니다.

비밀번호 찾기

- 가입 이메일을 입력하면 sendPasswordResetEmail.
- Firebase가 재설정 링크 메일을 보냅니다. 앱은 새 비밀번호를 직접 받지 않습니다.
- 메일 제목/본문은 콘솔 템플릿 “wishkit 비밀번호 재설정”.

이메일 찾기

- 사용자는 비밀번호가 아니라 아이디를 기억할 때가 많습니다. 아이디로 가입 이메일을 알려줍니다.
- users 에서 handleLower == 입력값 인 문서를 1개 찾습니다.
- 이메일은 그대로 보여주지 않습니다. k***m@gmail.com 처럼 가운데를 가립니다. (_maskEmail)
- 프로필이 공개 읽기인 이유 중 하나가 바로 이 찾기 기능입니다. 비밀번호는 절대 내려주지 않습니다.

왜 아이디로 이메일을 찾나

아이디는 친구에게 보여주는 공개 이름표입니다. 이메일은 로그인 키입니다. 아이디가 사람마다 유일해야, “@mina 로 가입한 메일”이 한 통으로 정해집니다. 아이디가 겹치면 이메일 찾기가 누구 메일인지 알 수 없고, 아래 12장의 팔로우 문제도 생깁니다.

10. 비밀번호 변경 · 로그아웃

비밀번호 변경 (마이페이지 → 비밀번호 수정하기)

- 현재 비밀번호로 재인증(reauthenticate)한 뒤 updatePassword.
- 새 비밀번호 6자 이상, 현재와 달라야 함.
- 너무 오래전에 로그인한 세션이면 requires-recent-login → “다시 로그인한 뒤 변경해 주세요.”

로그아웃

Firebase signOut 후 앱 메모리를 게스트 상태로 비웁니다. 위시리스트·친구 목록은 기기에 남아 보이지 않습니다. 같은 계정으로 다시 로그인하면 Firestore에서 다시 불러옵니다.

11. 계정 탈퇴

마이페이지 메뉴 → 탈퇴하기. 경고문(“되돌릴 수 없어요”)과 비밀번호 확인 후 진행합니다.

1. 재인증	2. 관계 끊기	3. 내 문서 삭제	4. 아이디 반납	5. Auth 삭제
비밀번호로 본인 확인	내가 팔로우한 사람, 나를 팔로우한 사람의 카운트까지 정리	tabs, products, following, followers, notifications, baskets, reviews	handles/ {아이디} 가 내 uid이면 삭제. 다른 사람이 그 아이디를 쓸 수 있게	user.delete(). 이후 같은 이메일로 다시 가입 가능

그림 2. 탈퇴 순서 — Auth보다 Firestore를 먼저 지운다

순서가 중요합니다. Auth 사용자를 먼저 지우면, 그 다음 Firestore 삭제가 권한 거부될 수 있습니다. 그래서 Firestore 트리와 handle 예약을 지운 뒤 마지막에 user.delete() 합니다. 콘솔에서 Auth만 지워 버린 낡은 데이터는, 같은 이메일로 다시 가입할 때 reclaimIdentityIfNeeded 가 치웁니다.

12. 아이디가 겹치면 안 되는 이유

팔로우 저장 자체는 uid 로 합니다. users/ {내uid}/following/ {상대uid} 와 users/ {상대uid}/followers/ {내uid}. 그런데 사람이 앱에서 친구를 고를 때는 uid가 아니라 아이디와 이름을 봅니다. 디렉터리·검색·알림·이메일 찾기도 handle 로 사람을 찾습니다.

겹치면 실제로 깨지는 것

- 누구를 팔로우했는지 화면에서 구분이 안 됩니다. 목록에 @mina 가 두 명이면, 버튼을 눌러도 기대한 사람이 아닐 수 있습니다.
- 이메일 찾기는 handleLower 로 문서 1개만 가져옵니다. 겹치면 다른 사람 메일이 가려진 채로 나올 수 있습니다.
- 알림에 fromHandle 이 붙습니다. 같은 아이디면 “누가 팔로우했는지” 알림이 거짓처럼 보입니다.
- 탈퇴 후 아이디 재사용이 위험합니다. 예약을 안 풀면 탈퇴한 아이디를 아무도 못 쓰고, 예약을 풀었는데 프로필에 같은 handleLower 가 남아 있으면 산 사람과 죽은 문서가 섞입니다.
- 친구 디렉터리는 users 컬렉션을 읽어 handle을 보여 줍니다. 유일하지 않으면 소셜 기능 전체가 “사람 = 아이디”라는 전제가 무너집니다.

그래서 한 일

장치	내용
handles/{소문자아이디}	전역 예약 테이블. 문서 ID가 아이디, 필드 uid가 주인. 한 아이디에 문서 하나
users.handleLower	프로필에도 소문자 아이디를 넣어 조회·이메일 찾기에 사용
가입 직전 assertHandleAvailable	이미 살아있는 프로필이 쓰면 “이미 사용 중인 아이디예요”
인증 후 트랜잭션 claim	동시에 같은 아이디를 치려는 두 가입을 한 명만 통과시킴
프로필 수정	아이디를 바꾸면 새 키를 차지하고, 내 것이던 옛 키를 삭제
탈퇴	내 uid가 적힌 handle 문서를 삭제해 아이디를 반납
같은 이메일 재가입	콘솔에서 Auth만 지운 잔여는 회수. 다른 사람 아이디는 절대 뺏지 않음

표 5. 아이디 유일성을 지키는 장치

한 문장으로 말하면, 팔로우는 uid로 저장하지만 사람은 아이디로 찾는다. 아이디가 유일해야 팔로우·팔로잉·친구 위시리스트·살까말까 보내기가 같은 사람을 가리킵니다.

13. Firestore 데이터 구조

경로	의미
users/{uid}	프로필. email, name, handle, handleLower, followers, following, avatarUrl
users/{uid}/tabs/{tabId}	위시리스트 탭. 신규는 ‘전체’만
users/{uid}/products/{productId}	찜한 상품
users/{uid}/following/{otherUid}	내가 팔로우하는 사람
users/{uid}/followers/{otherUid}	나를 팔로우하는 사람
users/{uid}/notifications/{id}	팔로우·살까말까·리뷰 알림
users/{uid}/receivedBaskets/{id}	친구가 보낸 살까말까
users/{uid}/sentBaskets/{id}	내가 보낸 살까말까

경로	의미
users/{uid}/reviews/{id}	상품 리뷰
handles/{handleLower}	아이디 → uid 예약. 예: handles/@mina

표 6. 계정과 소셜이 쓰는 Firestore 경로

Firebase 프로젝트 ID는 softstudio-wishlist-app 입니다. iOS 번들 ID는 com.softstudio.wishlist. 설정 파일은 flutter_app/lib/firebase_options.dart 입니다.

14. 보안 규칙

파일: wishlist-appversion2/firestore.rules . 배포는 firebase deploy --only firestore:rules . 규칙을 안 올리면 알림·장바구니 공유는 실패할 수 있지만, 앱은 그 실패로 로그인을 막지 않습니다.

- users 읽기는 누구나 가능. 친구 발견과 아이디로 이메일 찾기(가린 값)를 위해.
- users 생성은 본인 uid만. 인증된 사용자가 자기 프로필만 만들.
- 팔로우 카운트는 예외적으로 상대 문서의 followers/following 숫자만 ± 1 할 수 있음. 이름·이메일은 못 바꿈.
- handles 생성은 자기 uid를 넣는 경우만. 수정/삭제는 주인, 또는 주인 프로필이 사라진 고아 문서, 또는 같은 이메일로 재가입한 사용자.
- 탈퇴·재가입을 위해, 같은 이메일이면 낡은 하위 컬렉션을 지울 수 있게 해 두었습니다.

15. 에러 메시지

Firebase 코드는 영어입니다. 화면에서는 한국어로 바꿉니다.

Firebase / 상황	사용자에게 보이는 말
email-already-in-use	이미 가입된 이메일이에요. 로그인으로 전환해 보세요.
invalid-email	이메일 형식이 올바르지 않아요.
weak-password	비밀번호가 너무 짧아요 (6자 이상).
user-not-found / wrong-password / invalid-credential	이메일 또는 비밀번호가 맞지 않아요.
network-request-failed	네트워크 연결을 확인해 주세요.
too-many-requests	요청이 너무 많아요. 잠시 후 다시 시도해 주세요.
아이디 중복	이미 사용 중인 아이디예요.
requires-recent-login (탈퇴·비번변경)	보안을 위해 다시 로그인한 뒤 진행해 주세요.

표 7. 자주 나오는 인증 오류

16. 주요 파일

파일	역할
lib/main.dart	Firebase 초기화, 로그인 강제 라우팅
lib/firebase_options.dart	프로젝트 키. YOUR_ 이면 미연결로 간주
lib/data/app_store.dart	register / login / 인증 / 탈퇴 / 세션
lib/services/account_repository.dart	AuthFirestoreStorage-handle 유일성
lib/screens/auth/login_screen.dart	로그인·회원가입 폼
lib/screens/auth/email_verification_screen.dart	메일 인증
lib/screens/auth/account_recovery_screen.dart	이메일·비밀번호 찾기
lib/screens/auth/signup_welcome_screen.dart	가입 축하
lib/screens/mypage/mypage_screen.dart	비번 변경, 로그아웃, 탈퇴
firebase.rules	읽기/쓰기 권한, handles 예약 규칙
firebase_email_templates.txt	인증·재설정 메일 문구

표 8. 로그인 관련 파일 지도

17. 아직 없는 것 · 운영 시 확인할 것

현재 없는 기능

- 구글 로그인, 애플 로그인, 카카오 로그인. (카카오는 살까말까 공유용 SDK만 연결)
- 전화번호 로그인, 익명 로그인.
- 이메일 찾기에서 전체 이메일 표시. 항상 마스킹합니다.

배포·콘솔에서 확인할 것

- Authentication → Sign-in method 에서 Email/Password 사용 설정.
- Authentication → Templates 에 인증 메일·비밀번호 재설정 문구 반영.
- firestore.rules 최신본 배포. 안 올리면 알림·살까말까 권한이 거부될 수 있음.
- Storage 규칙도 아바타·리뷰 사진 업로드에 필요.
- 콘솔에서 사용자만 지우면 Firestore 잔여가 남을 수 있음. 앱 탈퇴 기능을 쓰는 편이 안전합니다.

기억할 문장 세 개

- 1) 자격 증명은 Auth, 사람 정보는 Firestore, 파일은 Storage.
- 2) 메일 인증 전에는 앱 계정이 없고, 인증 후에 아이디를 예약한다.
- 3) 아이디는 전역에서 하나. 겹치면 팔로우와 계정 찾기가 사람을 구분하지 못한다.

이 문서는 2026-08-14 기준 wishlist-appversion2 Flutter 앱 코드와 일치합니다. 소셜 로그인을 붙이거나 아이디 규칙을 바꾸면 이 설명서도 함께 고치면 됩니다.